

One identity, three standards: who does what

The old ONCHAINID was one big contract that did everything. Version 3 keeps the exact same functions on the outside, but splits the work between one small account contract and a few shared helper contracts (called modules). This page shows which contract does what. Each standard has its own color, so you always know which spec you are looking at.

● ERC-734 · keys: who may act

● ERC-735 · claims: what is attested

● ERC-7579 · modules: how the account is built

01 The three standards, in plain words

If you remember nothing else, remember these three sentences. Every line of code in this repo serves one of these three jobs.

ERC-734

The keyring

A list of **keys**. Each key has a **purpose** (MANAGEMENT, ACTION, CLAIM_SIGNER, ...). It answers one question: **"is this key allowed to do that?"** It also defines a two-step queue: one key *asks* (`execute`), another key *allows* (`approve`).

ERC-735

The badge wallet

A list of **claims**: signed statements about the identity, like "KYC passed" or "accredited investor". Each claim has a **topic** and an issuer who signed it. It answers: **"what has been said, and signed, about this identity?"**

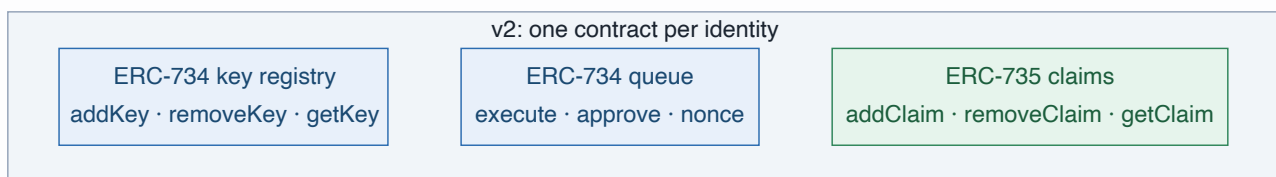
ERC-7579

The motherboard

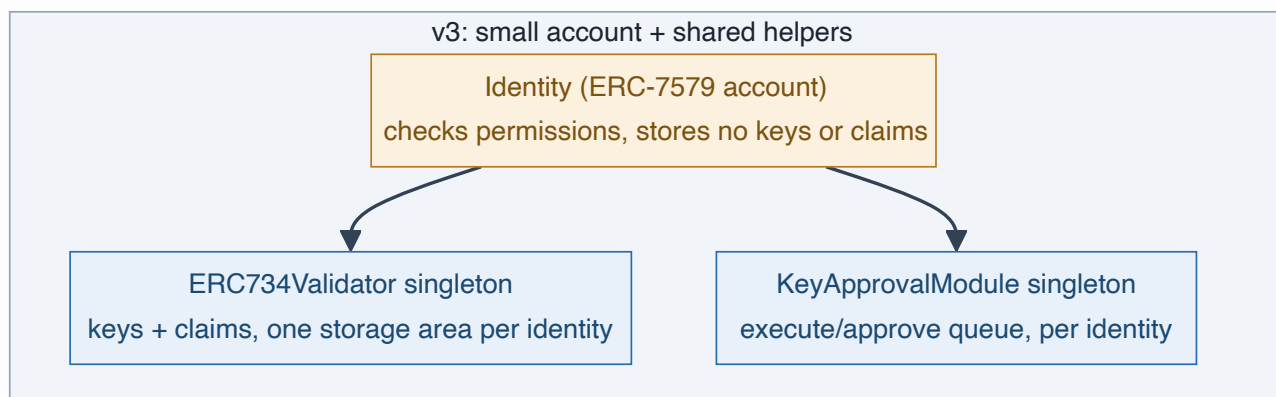
A **smart account built from parts**: a small core plus plug-in modules. **Validators** check signatures, **executors** send calls out, and **fallback handlers** answer functions the core doesn't have. ERC-4337 support means it also works with the EntryPoint.

02 What changed: v2 monolith → v3 modular

From the outside, nothing changed. The identity still answers every ERC-734 and ERC-735 function, and `supportsInterface` still says yes to all three standards. What changed is *where the code and the data live*. In v2, everything was inside one contract. In v3, the account only checks permissions; the keys, claims, and queue live in shared helper contracts. Each helper is deployed once and serves every identity, keeping a separate storage area per identity.



v2: everything in one contract, one storage layout.

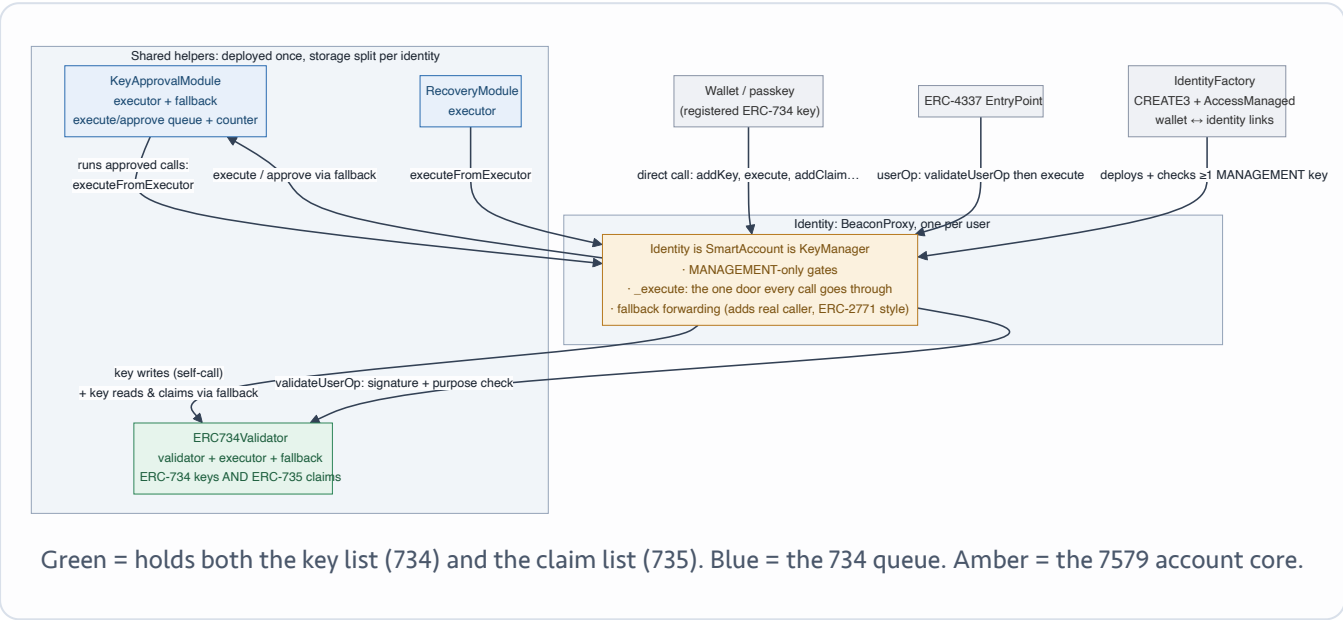


v3: the account checks, the helpers store.

CONCERN	V2 (ONE CONTRACT)	V3 (THIS BRANCH)
734 key list	Stored inside each Identity	Stored in the <code>ERC734Validator</code> helper, which is deployed once and keeps a <code>registries[account]</code> entry per identity. Its address is baked into the Identity code as an immutable , so it cannot be swapped out ("enshrined").
734 adding / removing keys	Code on the Identity	<code>KeyManager</code> is now just a thin front door with no storage : it checks the caller has MANAGEMENT, then passes the write on to the helper.
734 reading keys	Code on the Identity	Not on the account at all. The account's 7579 fallback (its "catch-all") forwards the call to the helper.
734 execute / approve queue	Counter + queue inside the Identity	Moved to <code>KeyApprovalModule</code> . It owns the counter, the queue, and the rules for what runs automatically.
735 claims	Stored inside each Identity	Live in the <i>same</i> <code>ERC734Validator</code> helper as the keys, reached through the fallback. If no handler is wired for a claim function, the call reverts (<code>ERC7579MissingFallbackHandler</code>).
7579 checking signatures	ECDSA only, hardcoded	<code>ERC734Validator</code> , acting as the validator module. It accepts ERC-7913 signers (a normal wallet, another contract via ERC-1271, or a passkey via WebAuthn) and checks the signer is a registered key with the right purpose.
7579 deployment	CREATE2 clones	<code>IdentityFactory</code> deploys a BeaconProxy through CREATE3, so an identity gets the same address on every chain . Wallet links are permanent, and unlinking is final.

03 The v3 map: every arrow you will audit

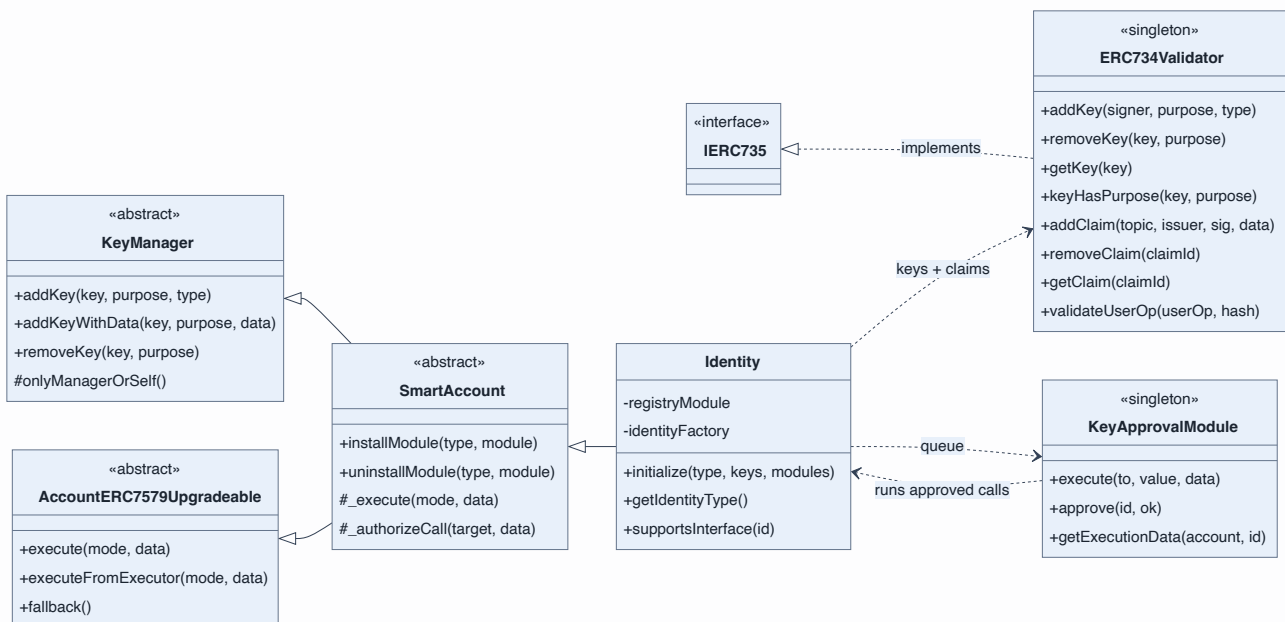
One diagram, whole system. The amber box is a single contract: `Identity` inherits `SmartAccount`, which inherits `KeyManager`. The two helpers below it are each deployed *once* and shared by every identity, with storage split per identity. That isolation is itself something to audit.



Built in, not just plugged in: the `ERC734Validator` address is an `immutable` inside the Identity code. Even if the module is uninstalled, `KeyManager` keeps reading from it, and uninstalling never deletes keys or claims. The data survives on purpose. Replacing this helper means deploying a new Identity implementation and upgrading the beacon.

UML class diagram: inheritance and wiring

Solid arrows are inheritance (the three account contracts compile into one). Dashed arrows are calls between deployed contracts.



`KeyManager`, `SmartAccount`, and `Identity` are one deployed contract. `registryModule` and `identityFactory` are immutables set at deploy time. The two modules are separate singleton deployments; `AccountERC7579Upgradeable` is OpenZeppelin's base account.

04 Who answers which function

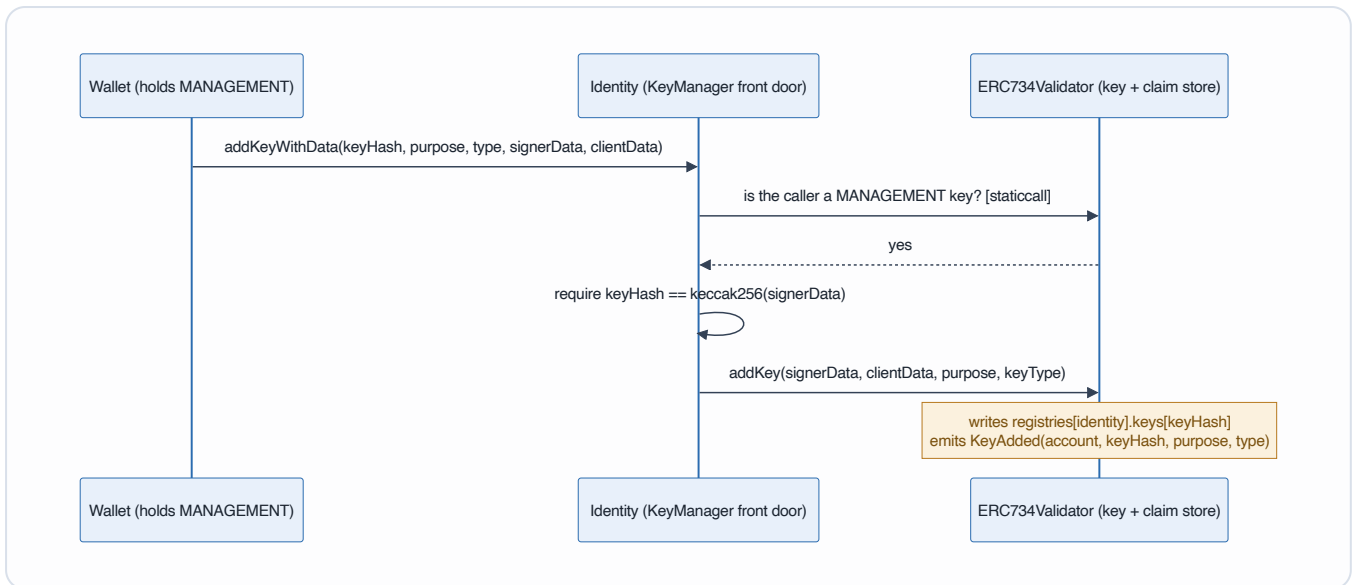
Callers only ever talk to the `Identity` address. This table is the dispatch truth: for each function, which contract really runs the code, how the call gets there, and who may call it.

FUNCTION(S)	STANDARD	CODE RUNS IN	HOW IT GETS THERE	WHO MAY CALL
addKey addKeyWithData removeKey	734	KeyManager → ERC734Validator	Exists on the account; the account passes the write on to the helper	A MANAGEMENT key, or the identity calling itself
getKey getKeyPurposes getKeysByPurpose keyHasPurpose	734	ERC734Validator	Not on the account. The fallback forwards it and adds the real caller's address (ERC-2771 style)	Anyone (read-only)
execute approve	734	KeyApprovalModule	Fallback forwards it; the module later calls back in through executeFromExecutor	Queue: any real purpose. Approve: MANAGEMENT for self-targets, ACTION for outside targets
addClaim removeClaim getClaim getClaimIdsByTopic	735	ERC734Validator	Fallback forwards it, to the same helper that holds the keys	CLAIM_SIGNER / CLAIM_ADDER rules, checked in the helper
installModule uninstallModule	7579	SmartAccount	Directly on the account	A MANAGEMENT key. Stricter than OZ's default (EntryPoint-or-self)
validateUserOp isValidSignature	7579	ERC734Validator (as validator)	The account hands validation to the installed validator	Signature must match a registered key with the right purpose
execute(mode, calldata) executeFromExecutor	7579	SmartAccount._execute	Called by the EntryPoint, the identity itself, or an installed executor	The one door every call goes through (see §06)

05 The flows: UML sequence, activity, and state diagrams

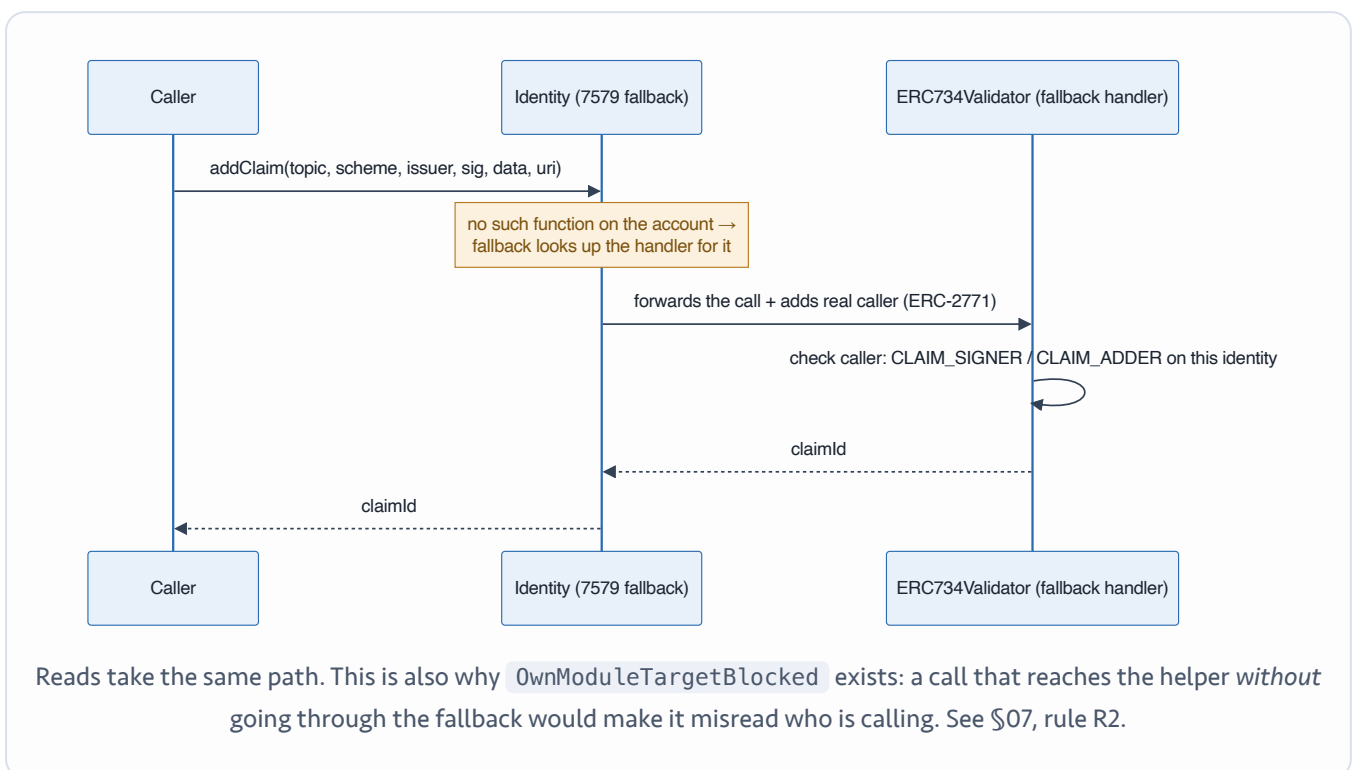
Flow A: adding a key (sequence diagram)

The account stores no keys. It checks the caller is allowed, then passes the write on. Note the helper plays two roles here: it answers "is the caller a manager?" *and* it stores the new key.



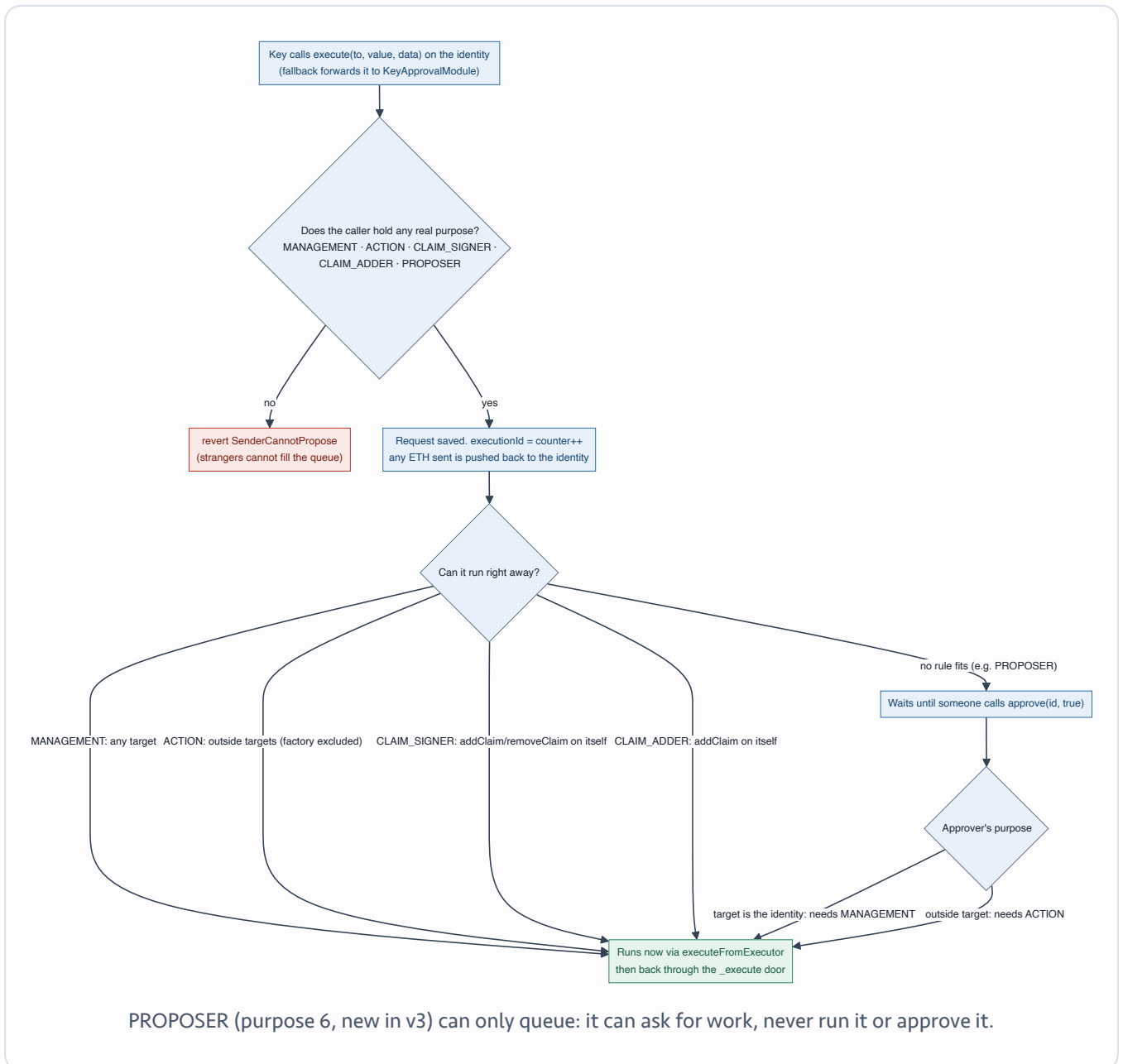
Flow B: reading or writing a claim (sequence diagram)

Claim functions don't exist in the Identity's bytecode. The call lands in the account's catch-all (the 7579 fallback), which forwards it to the helper registered for that function and adds the real caller's address to the end of the call data.



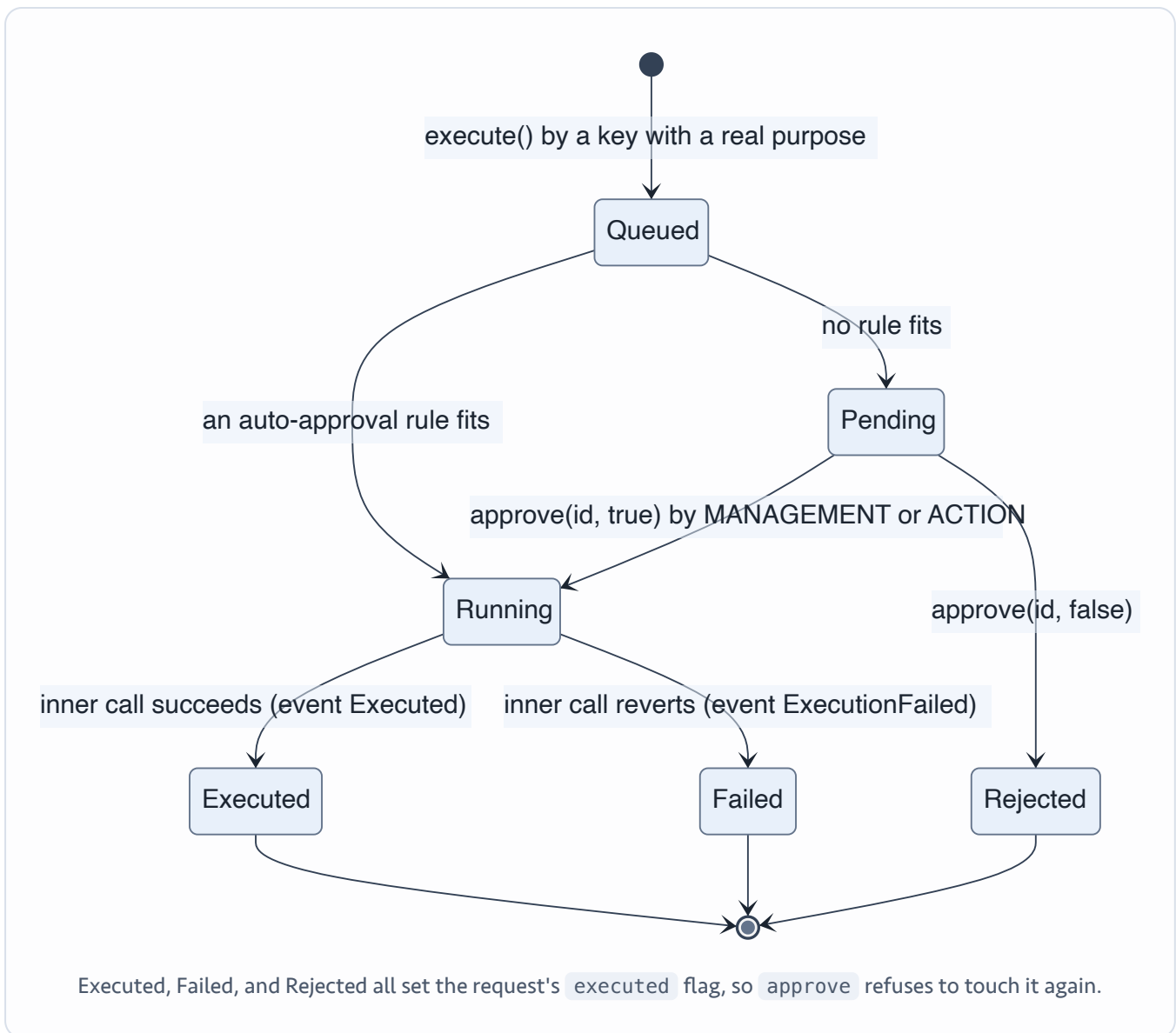
Flow C: the execute / approve queue (activity diagram)

The classic ERC-734 flow, now fully owned by `KeyApprovalModule`. Two separate doors: *getting into the queue* is easier than *getting the call to run*.



Life of one queued request (state diagram)

The same queue, seen from one request's point of view. There is no way back from a final state: a rejected or failed request stays closed, and the work must be queued again as a new request.



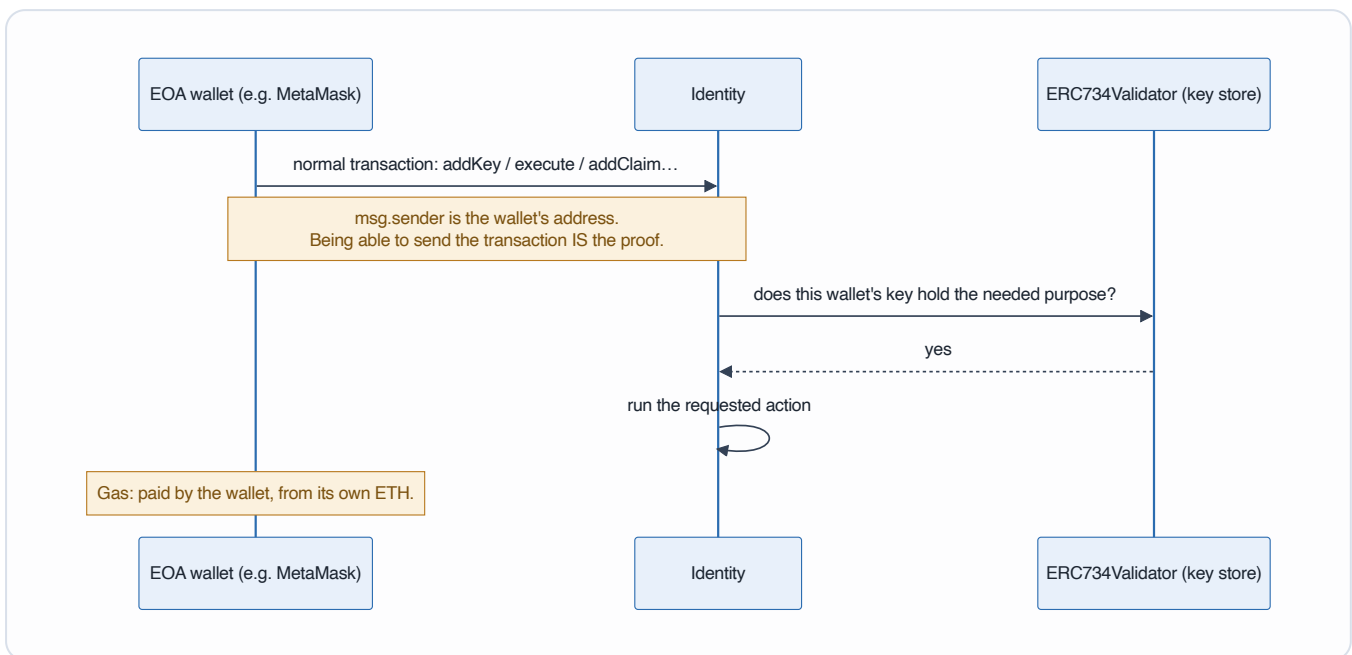
Flow D: same identity, four ways to sign and pay

The identity accepts several kinds of signers (ERC-7913). What changes between them is *who signs*, *who sends the transaction*, *who pays the gas*, and *who checks the signature*. This table is the summary; the three sequence diagrams below walk through each path.

PATH	WHO SIGNS	WHO SENDS THE TRANSACTION	WHO PAYS GAS	WHO CHECKS THE SIGNATURE
D1 · EOA, direct call	The wallet (normal transaction)	The wallet itself	The wallet, from its own ETH	Nobody. Sending the transaction is the proof; the account only checks the wallet's key purpose
D2 · Passkey, ERC-4337	The device (Face ID / Touch ID, P-256)	A bundler, through the EntryPoint	The identity, from its own ETH	<code>ERC734Validator</code> , which asks the <code>WebAuthn</code> verifier contract
D3 · Gasless, paymaster	EOA or passkey (either works)	A bundler, through the EntryPoint	The paymaster. The user needs zero ETH	<code>ERC734Validator</code> , plus the paymaster's own check
D4 · Contract signer, ERC-1271	Another contract (e.g. a multisig)	Any of the above paths	Depends on the path used	<code>ERC734Validator</code> , which asks the signer contract's <code>isValidSignature</code>

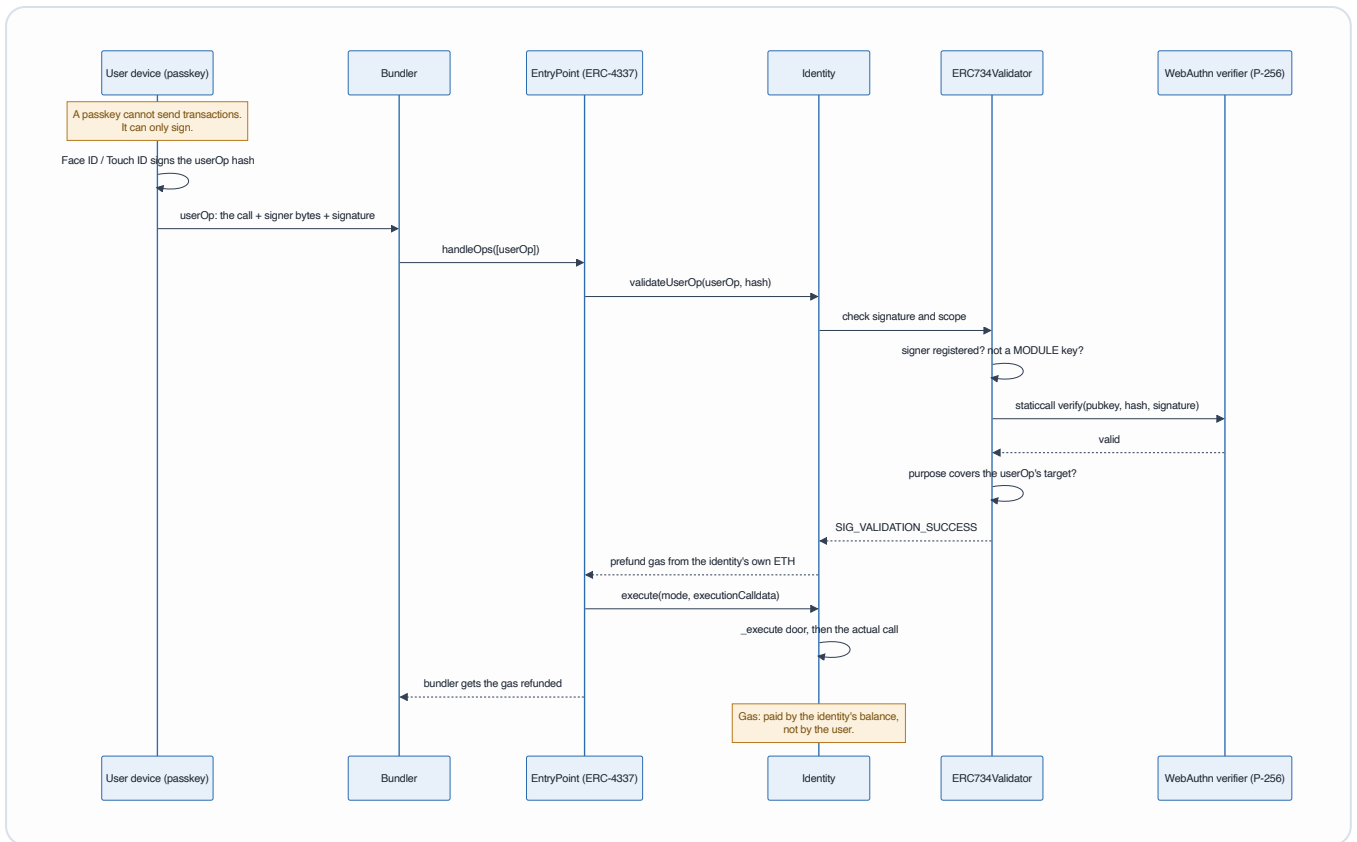
D1: EOA wallet, direct transaction (sequence diagram)

The simplest path. The wallet is registered as a key, so `msg.sender` is all the proof needed. There is no signature to verify on-chain.



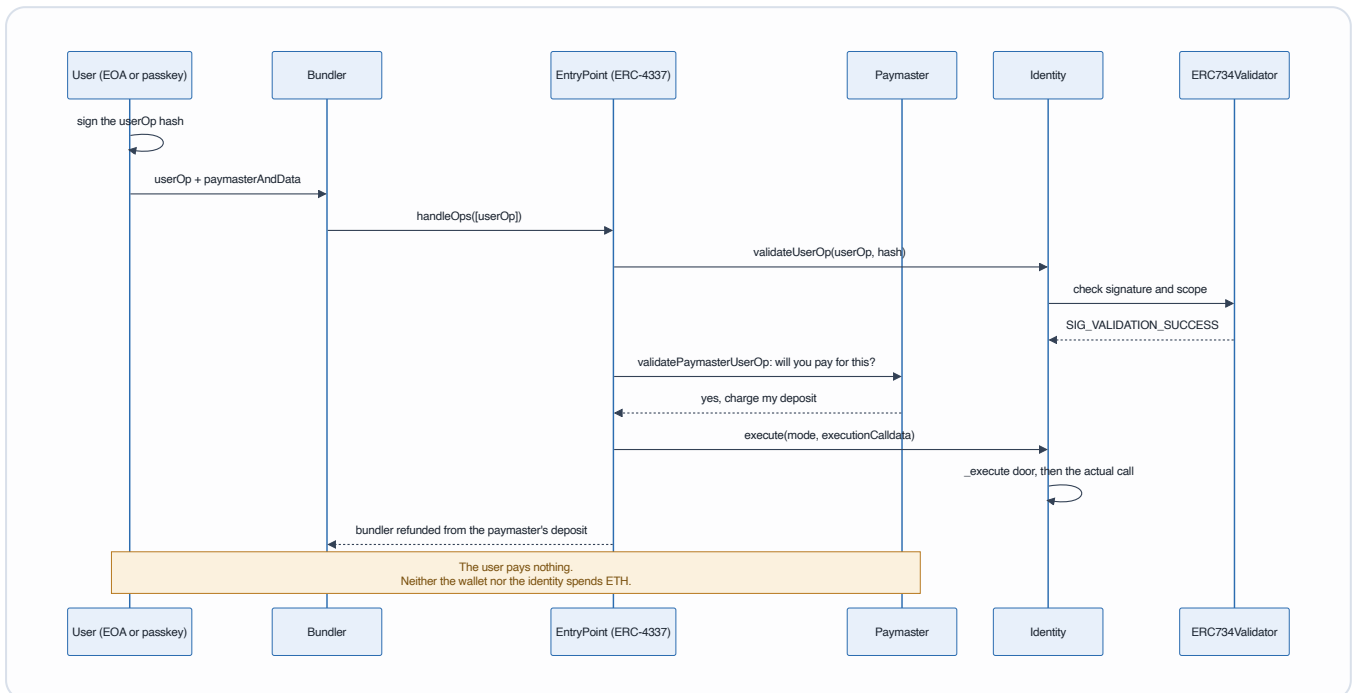
D2: passkey (WebAuthn), through ERC-4337 (sequence diagram)

A passkey has no address, so it cannot send a transaction. It can only sign. The signed request (a `userOp`) travels through a bundler and the EntryPoint, and the identity pays its own gas. The signer bytes stored in the key registry are `verifier || pubkey`: the first 20 bytes name the P-256 verifier contract, the rest is the public key.



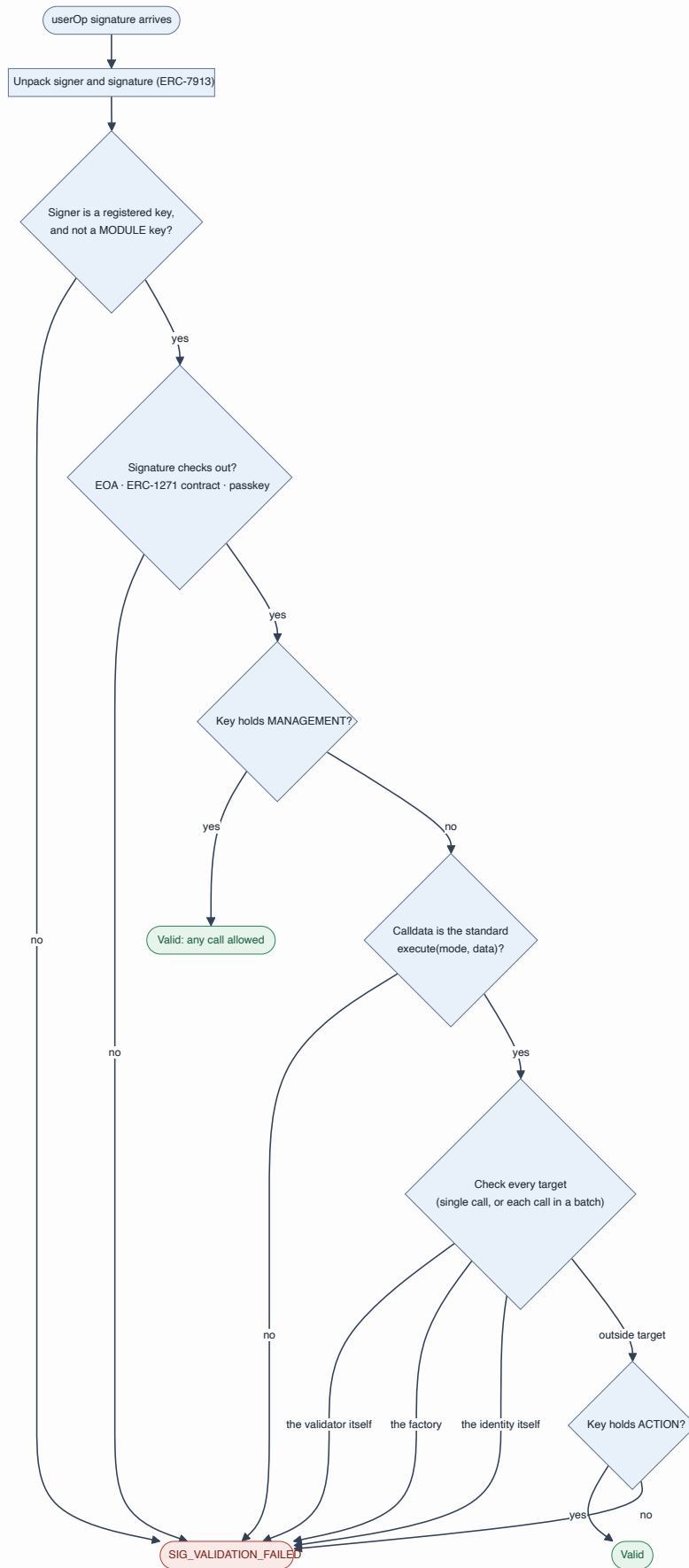
D3: gasless transaction with a paymaster (sequence diagram)

Same rails as D2, but a paymaster offers to pay. The user (EOA or passkey) signs and needs zero ETH. Two validations run: the identity checks the signature, the paymaster checks it is willing to sponsor this call.



How the validator decides (activity diagram)

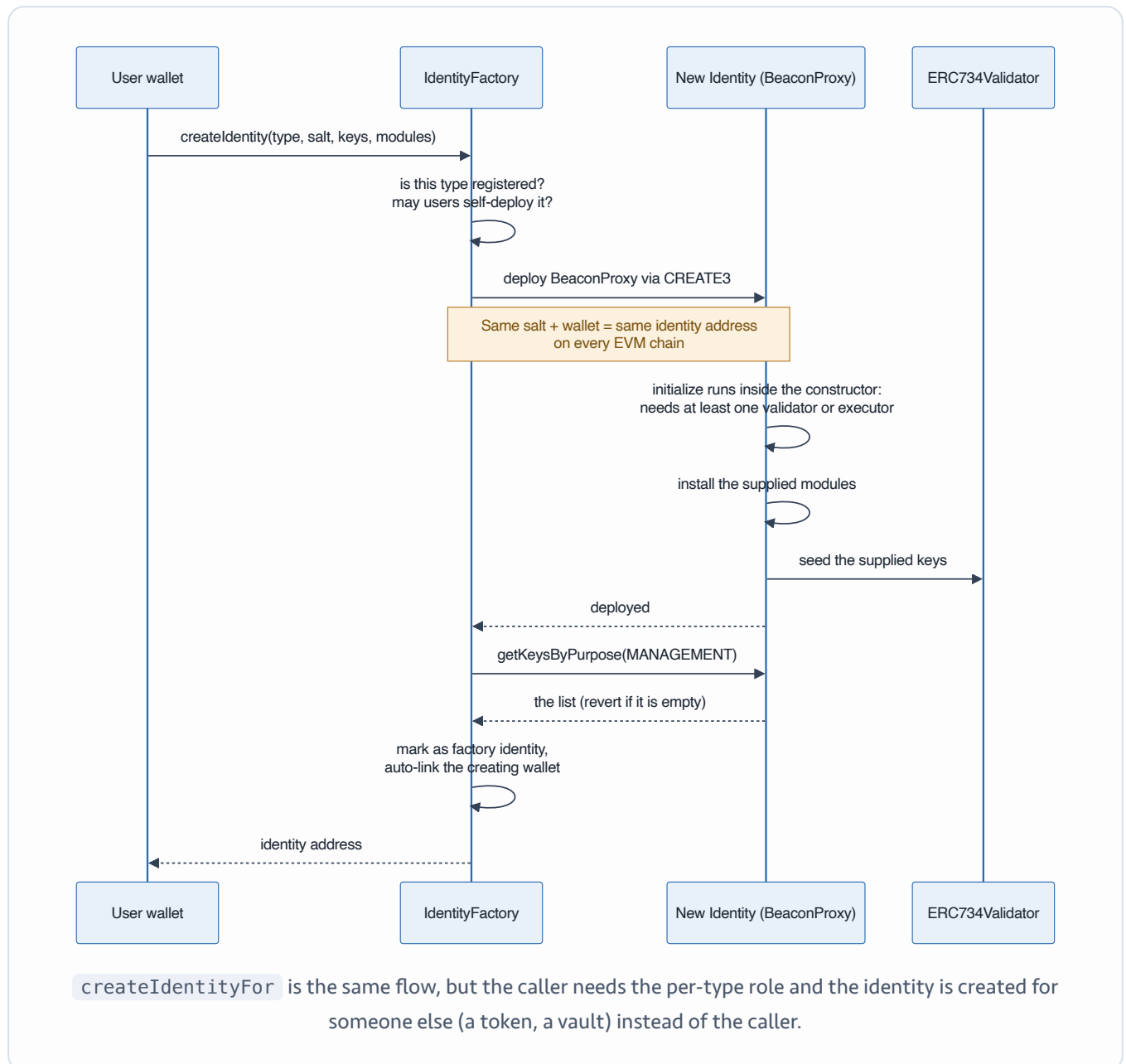
The exact decision tree inside `ERC734Validator` for a userOp. Note what a non-MANAGEMENT key can never reach: the identity itself, the validator, and the factory.



The self-guard on the validator matters most: without it, an ACTION key could call `execute(validator, addKey(attacker, MANAGEMENT))` as an "outside" call and promote itself. A batch is gated by its weakest call.

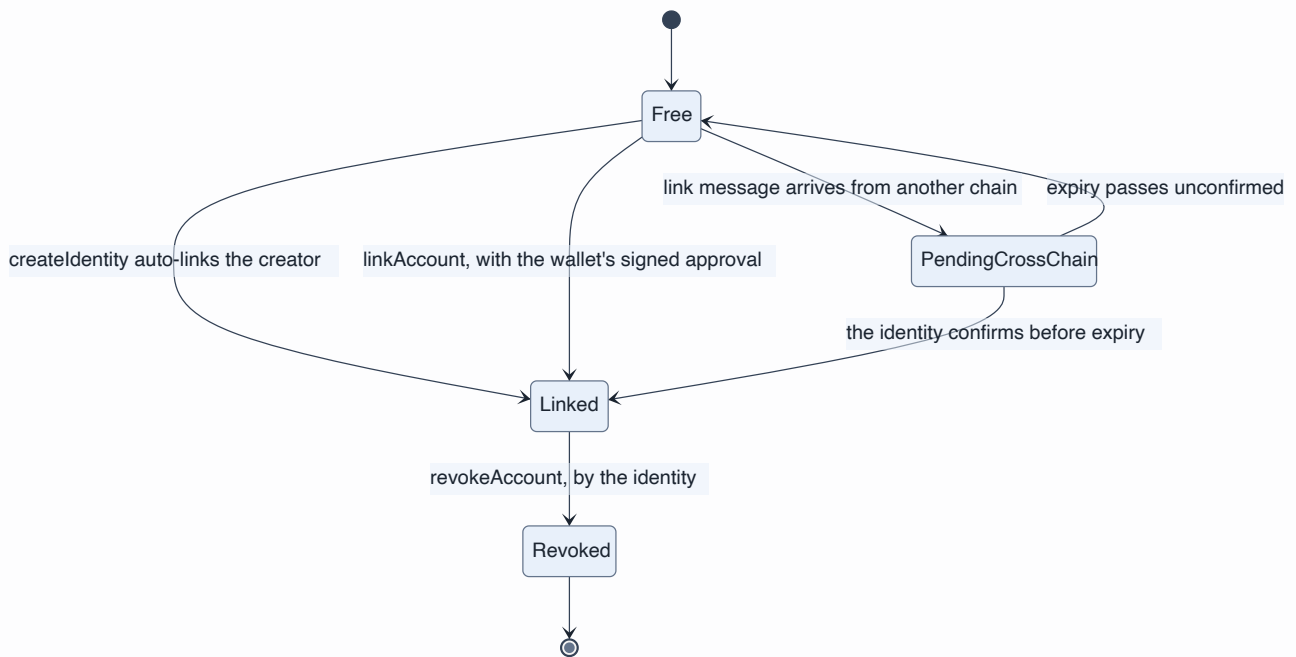
Flow E: how an identity is born (sequence diagram)

The factory is the only door for creating identities. Setup runs entirely inside the proxy's constructor, so there is never a moment where a half-built identity is callable from outside. The factory then double-checks the result before handing it over.



Wallet ↔ identity binding (state diagram)

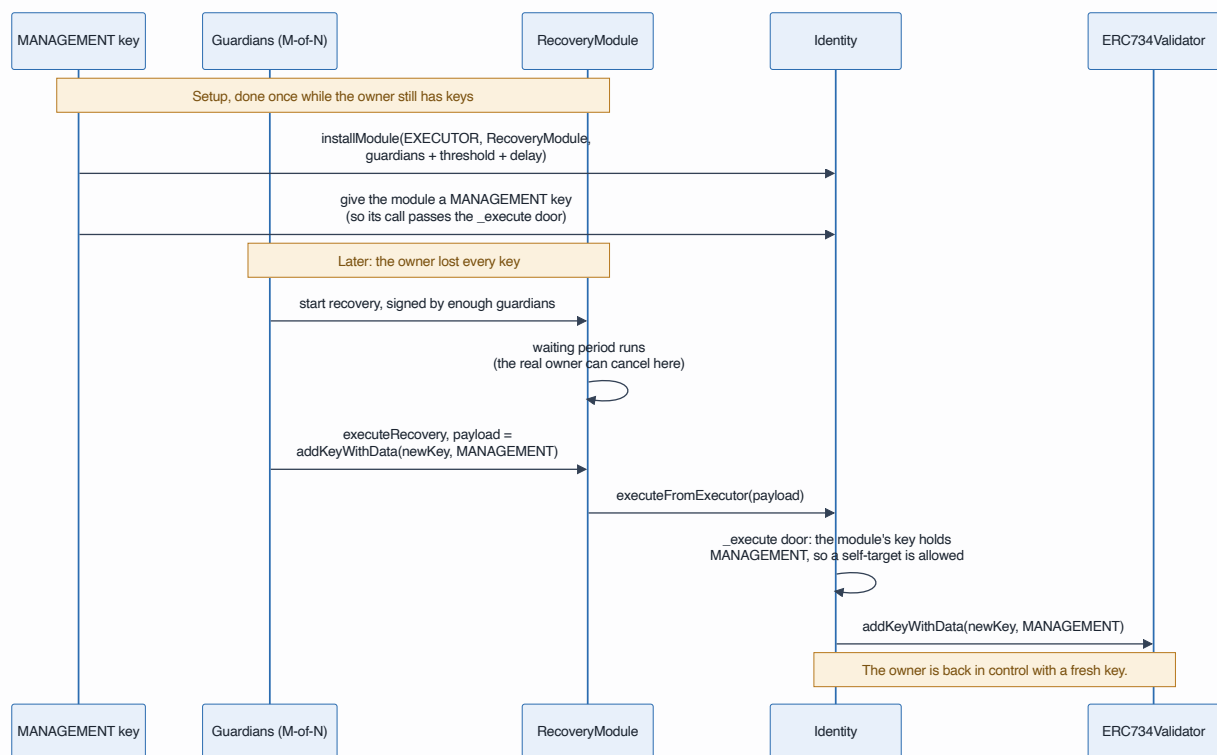
The factory keeps a registry of which wallet belongs to which identity. Two hard rules: a linked wallet is *stuck* to its identity (it can never link to a second one), and a revoked wallet is *done* (it can never be linked again, anywhere).



ASSET and SMART_CONTRACT identities are bound to one contract at deploy and can neither link extra wallets nor revoke: their bound contract cannot sign. Cross-chain messages are only accepted from gateways the admin has whitelisted.

Flow F: social recovery, when all keys are lost (sequence diagram)

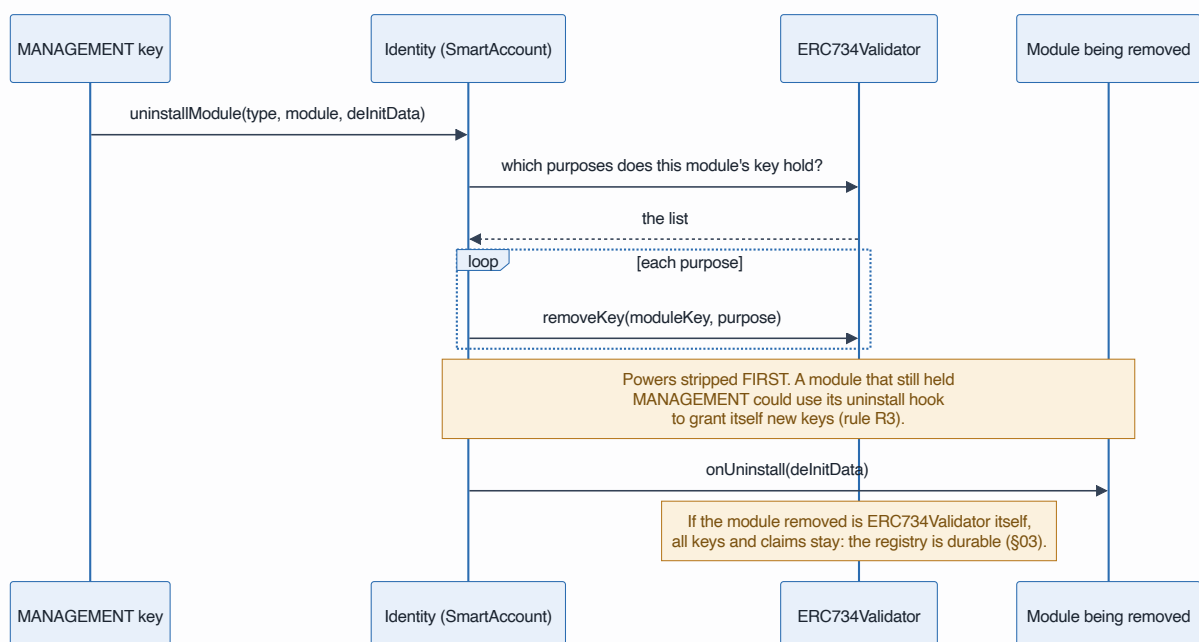
`RecoveryModule` is OZ's audited-design social recovery executor, unchanged. Guardians (friends, hardware keys, a service) can together install a fresh MANAGEMENT key after a waiting period. The real owner can cancel during the wait.



Audit note: this module holds authority that can replace MANAGEMENT keys. It is pinned to a specific OZ commit; the upstream repo is not yet tagged as audited.

Flow G: uninstalling a module safely (sequence diagram)

The order here is a security decision (rule R3): the module's powers are taken away *before* its own uninstall code gets to run.



06 Key purposes: the whole permission model in one table

Six purposes (library `KeyPurposes`). `MANAGEMENT` passes every check. This table merges the queue rules and approval rules from `KeyApprovalModule` with the admin gate from `SmartAccount` .

PURPOSE	PUT A REQUEST IN THE QUEUE	RUNS RIGHT AWAY WHEN...	APPROVE A WAITING REQUEST	ADMIN: MODULES & KEYS
1 <code>MANAGEMENT</code>	yes	always, any target	yes, any target	yes
2 <code>ACTION</code>	yes	outside targets only, never the identity itself or the factory	outside targets only	no
3 <code>CLAIM_SIGNER</code>	yes	only <code>addClaim</code> / <code>removeClaim</code> on itself	no	no
4 <code>ENCRYPTION</code>	no (off-chain use only)	never	no	no
5 <code>CLAIM_ADDER</code>	yes	only <code>addClaim</code> on itself (cannot remove)	no	no
6 <code>PROPOSER</code> (new)	yes, and that is all it can do	never	no	no

Modules can be keys too: a module installed with a purpose gets a `MODULE`-type key. For example, `KeyApprovalModule` holds `MANAGEMENT` so its approved self-targeted calls pass the `_execute` door. Inside that door, an executor's key needs `MANAGEMENT` to call the identity or the factory, and `ACTION` for anything else.

07 Where the data lives

All storage uses `ERC-7201` named slots. The proxy itself holds almost nothing. The interesting data sits in the shared helpers, filed under each identity's address. Making sure one identity cannot touch another's data is an audit target.

CONTRACT	ERC-7201 NAMESPACE	WHAT'S INSIDE
<code>7579</code> Identity proxy	<code>onchainid.identity.metadata</code> <code>onchainid.accountERC7579.storage</code> (OZ)	The identity type; which modules are installed and which handler answers which function. No keys, no claims.
<code>734</code> + <code>735</code> <code>ERC734Validator</code>	<code>onchainid.validators.erc734-validator</code>	<code>registries[account]</code> holds the keys (purposes, signer bytes, metadata) <i>and</i> the claims (by id, by topic, revoked digests). Survives uninstall.
<code>734</code> <code>KeyApprovalModule</code>	plain mapping <code>_state[account]</code>	The request counter and the queued requests, per identity.
<code>7579</code> <code>IdentityFactory</code>	<code>onchainid.IdentityFactory</code>	Wallet ↔ identity links (permanent; unlinking is final), per-type rules, salts.

08 Rules the design depends on

If any of these can be broken, that is a finding. Each one is deliberate, and the in-source natspec explains the reasoning.

R1 Every call goes through one door.

Every dispatched call, whether from a userOp, a manager, or an executor, passes through `SmartAccount._execute`. `DELEGATECALL` and unknown call types revert. A malformed payload (under 52 bytes) reverts before it could slip past the checks.

contracts/SmartAccount.sol: `_execute`, `_authorizeCall`

R2 No dispatched call may target the account's own modules.

Modules expect calls to arrive through the fallback, which adds the real caller's address. A raw `execute(module, ...)` skips that step, so the module would think the identity itself is calling and grant too much. Blocked with `OwnModuleTargetBlocked`.

contracts/SmartAccount.sol: `_authorizeCall`

R3 A module loses its purposes before its uninstall hook runs.

A module that still held `MANAGEMENT` could use its `onUninstall` callback to give itself new keys. So `_uninstallModule` strips every purpose first, then uninstalls.

contracts/SmartAccount.sol: `_uninstallModule`

R4 The last `MANAGEMENT` key cannot be removed.

Enforced in the helper (`CannotRemoveLastManagementKey`), so no path (front door, fallback, or queue) can lock an identity out by deleting all its manager keys.

contracts/modules/validators/ERC734Validator.sol: `removeKey`

R5 A key hash must match its signer bytes.

`keyHash == keccak256(signerData)` is required on registration (checked in the front door and re-derived in the helper). Nobody can register one key hash while attaching a different signer's material.

contracts/KeyManager.sol: `_addKeyWithData`

R6 Setup happens once, with no outside window.

`initialize` runs inside the proxy's constructor, makes no calls to other contracts, and never writes a temporary bootstrap key anywhere. The contract requires at least one validator or executor module; the factory then checks at least one MANAGEMENT key exists. The implementation contract locks its own initializer.

contracts/Identity.sol: constructor, `initialize`

R7 Installing modules needs MANAGEMENT, deliberately stricter than OZ.

OZ's default allows the EntryPoint or the account itself; here only a MANAGEMENT key (or a self-call) may install. Both old paths already needed MANAGEMENT, so this tightens, not loosens. But note: *installing a validator hands it full userOp authority*. A validator that doesn't limit its signers can reach admin calls like `addKey`. Only MANAGEMENT can install one.

contracts/SmartAccount.sol: `installModule` natspec

R8 One identity must never touch another's data.

`ERC734Validator` and `KeyApprovalModule` serve every identity from one deployment, filed under `msg.sender` (the account). Any path that lets identity A write into identity B's keys, claims, or queue is a critical finding. The queue also never holds ETH: any value sent is pushed straight back to the identity.

contracts/modules/validators/ERC734Validator.sol · contracts/modules/executors/KeyApprovalModule.sol